NYC Urban Mobility Explorer: Technical Report

1. Problem Framing and Dataset Analysis

For this project, we used the New York City yellow taxi data from January 2019. Every row in the data is one taxi trip. It tells us when the trip started and ended, where it started and ended, how far it went, how many people were in the taxi, and how much the fare was. We wanted to understand how people move around the city and show it in a simple dashboard.

The data was not perfect, so we faced some problems. Some fields were empty, like the number of passengers. Some trips had zero distance or a negative fare, which does not make sense. Some trips ended before they started, and some were not even in January. A few trips also showed speeds that no taxi could really drive in the city.

We made some simple rules when cleaning the data. We only kept trips with a distance and fare above zero, and a dropoff time after the pickup time. We kept passenger counts from one to six and removed trips outside January 2019. We chose to delete bad rows instead of guessing the missing values, because guessing could make our charts wrong. We also saved a log of the removed rows so anyone can check our work.

One thing that surprised us was that some trips had zero distance but still had a real fare. This showed us that some records were cancelled or saved by mistake. Because of this, we added a distance check to our cleaning step so these trips would not change our results.

2. System Architecture and Design Decisions

Our system has three parts. The first part is the frontend, which is the web page people see. We built it with HTML, CSS, and JavaScript, and we used Chart.js for charts and Leaflet for the map. The second part is the backend, which we built with Python and Flask. It takes requests from the web page, asks the database for the data, runs our own algorithm, and sends the answer back. The third part is the MySQL database, which stores the cleaned trip data. The web page talks to the backend, and the backend talks to the database. The web page never talks to the database directly. Our database design picture is linked in our README, and we added it to this report.

We chose these tools because they are simple and fit our needs. HTML, CSS, and JavaScript work in any browser. Chart.js and Leaflet made it easy to build charts and maps. Python is great for cleaning data, and Flask is small and quick to set up. MySQL works well because our data fits nicely into tables and it is good at filtering and grouping.

We also made some trade-offs. We cleaned the data before putting it in the database, which keeps the database small and fast, but it means the raw data is not stored there. Using separate tables saves space, but it needs joins, which are a bit slower. Flask is easy to use, but it is not made for very big traffic. We also used only one month of data so our app stays fast.

3. Algorithmic Logic and Data Structures

We needed to find the busiest zones in the city, but we were not allowed to use built-in sorting or counting tools. So we wrote our own code for it.

First, we count the trips for each zone. We go through the trips one by one and keep a simple list of zone names and their counts. Each time we see a zone, we add one to its count. Second, we find the top zones. We keep a short list of the best zones so far. If the list is not full, we place the new zone in the right spot. If the list is full, we compare the new zone with the smallest one in the list. If the new zone is busier, we remove the smallest one and put the new zone in its correct spot. At the end, our list has the busiest zones in order.
For time, counting grows with the number of trips, so more trips means more work. Finding the top zones grows with the number of zones times the size of our short list. This is faster than sorting everything when we only need a few top results. For space, we only need one count for each zone plus our short list, so it does not use much memory.
4. Insights and Interpretation
Insight one is about the busiest hours. We counted the trips for each hour of the day using a database query and showed the result in a bar chart. The chart shows that taxi use goes up in the morning and evening rush hours. This matters for urban mobility because it shows when roads are most crowded and when the city needs the most taxis.
Insight two is about the busiest zones. We used our own algorithm to find the top pickup zones and showed them on the map. The busiest zones are mostly in the center of the city and near the airports. This shows where the city needs more pickup spots and better public transport.
Insight three is about distance, fare, and tips. We compared the average fare and tip with the trip distance using a line chart. Longer trips cost more, but most trips are short. This tells us that people mostly use taxis for quick trips inside the city.

5. Reflection and Future Work

We had some technical challenges. The data file was very large, so cleaning it took time. Some database queries were slow, and connecting the web page to the backend was hard at first. We fixed the slow queries by cleaning the data first and doing the filtering inside the database.

We also had some team challenges. It was hard to share the work fairly, and combining everyone’s code in Git sometimes caused conflicts. To stay organized, we used a shared worksheet, held group meetings, and made a shared database diagram.